Attack-Graph-Based Cloud Misconfiguration Risk Scoring

Course: Information, Security & Privacy (ISP) – 7th Semester
Mentor: Sezal Rana
Team: 7 members
Duration: 8 weeks (Week 1-8)
Repository: https://github.com/abhimnyu09/Attack-Graph-Based-Cloud-Misconfiguration-Risk-Scoring

---

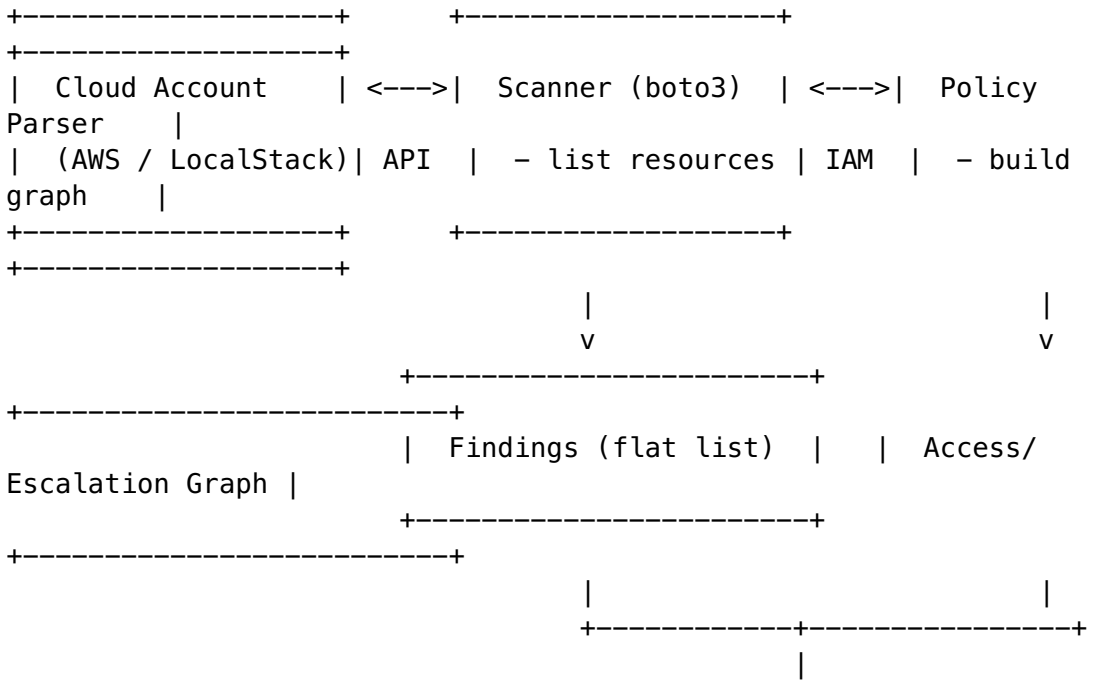
1. Problem Statement

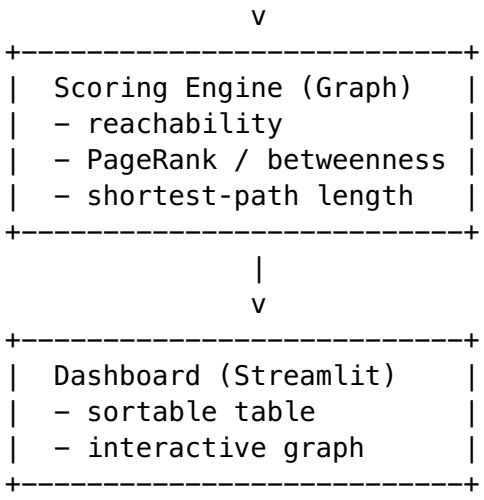
Traditional Cloud-Security-Posture-Management (CSPM) tools emit a flat list of misconfigurations (public S3 bucket, over-permissive IAM role, open security group, missing MFA, …).
All findings look equally urgent, so analysts waste time on low-risk items while a short **privilege-escalation chain** to full admin stays hidden.

Our goal: Build a pipeline that not only lists misconfigurations but also **ranks them by real exploitability** using an IAM-access / privilege-escalation graph.

---

2. High-Level Architecture





All components are containerised; a single `docker compose up --build` brings the whole stack up.

---

3. Deliverables (what we must hand-in)

#	Artefact	Format
1	Seeded test cloud account (≈ 20 misconfigs)	Terraform + `misconfig_catalog.json`
2	Flat-rule scanner	Docker image → `findings.jsonl`
3	IAM access / escalation graph	`graph.graphml` (NetworkX)
4	Scoring engine (reachability, PageRank, hop-count)	`scored_findings.jsonl`
5	Interactive dashboard (Streamlit + `streamlit-agraph`)	UI at `localhost:8501`
6	Evaluation report (Precision@k, MAP, statistical test)	`final_report.pdf` + plots
7	10-min live demo + 10-slide deck	`slides.pdf`, `demo.sh`
8	Reproducible repo (CI, `docker-compose up`)	Public GitHub repo

---

4. Work Plan (8 weeks, 7 people)

Week	Phase	Owner(s)	Exit Criteria
0	Repo & CI bootstrap	Coordinator	GitHub repo, protected `main`, CI green
1-2	**Seed cloud account** – Terraform with ~20 misconfigs	Cloud-team (2)	`make seed` creates resources, `misconfig_catalog.json` (≈ 20 entries)

Week	Phase	Owner(s)	Exit Criteria
2-3	**Scanner** – real `boto3` enumeration	Scanner-team (2)	`make scan` → `findings.jsonl` matches catalog
3-4	**Graph builder** – parse IAM policies → NetworkX graph	Graph-team (2)	`make graph` → `graph.graphml`
4-5	**Scoring engine** – reachability, PageRank, hop-count	Scoring (1) + Graph	`make score` → `scored_findings.jsonl` with `risk_score` & `attack_path`
5-6	**Dashboard** – Streamlit table + interactive graph	UI-team (2)	`docker compose up` shows UI at `localhost:8501`
6-7	**Evaluation** – Precision@k, MAP, Wilcoxon test	Analyst (1)	`eval/results.csv`, plots, statistical significance
7-8	Documentation, demo rehearsal, final polish	All	`final_report.pdf`, `slides.pdf`, `demo.sh`, tag `v1.0-eval`

Parallelism: Infra & scanner can run together; graph starts once seed finishes; scoring & UI develop in parallel after graph format is frozen. Critical path ≈ 5 weeks → 3 weeks buffer.

---

5. Current Progress (as of Friday evaluation)

Component	Status	Remarks
Documentation	✅ Complete – all markdown chapters + PDF (`PROJECT_REPORT.pdf`)	
Repo scaffolding	✅ `docker-compose.yml`, `Makefile`, `README.md`, CI workflow	
CI / Branch protection	✅ GitHub Actions (lint, pytest, hadolint via Docker, Docker build, smoke test)	
Infrastructure (Terraform seed)	✅ Minimal, working seed (~18 misconfigs)	

Component	Status	Remarks
	that applies cleanly on LocalStack 1.4.0	
Scanner	🟡 Placeholder – dummy writer only	
Graph builder	🟡 Placeholder – static 2-node graph	
Scoring engine	🟡 Placeholder – constant scores	
UI / Dashboard	✅ Works with dummy data (Streamlit + `streamlit-agraph`)	
Evaluation & report	⏳ Not started	
Branch protection & PR workflow	✅ Enforced on `main` (PR-only, 1 approval, CI gate, linear history)	

What works locally today

```
# start LocalStack (v1.4.0) — all services healthy
docker start localstack

# export env vars for Terraform / scanner
export ENDPOINT_URL=http://127.0.0.1:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1

# run the whole dummy pipeline
make seed   # creates ~18 real resources in LocalStack (IAM,
        DynamoDB, KMS, Lambda, SG, etc.)
make scan && make graph && make score && make ui
# UI → http://localhost:8501  (two-record dummy dashboard)
```

All CI checks (flake8, pytest, hadolint via Docker, Docker build, smoke test) are green on `main`.

---

6. Next Phases & Immediate Next Steps

Next Ticket	Branch	First Commit
Scanner	`feat/scanner`	Replace `scanner/main.py` with real `boto3` enumeration of the seeded account → `findings.jsonl`
Graph Builder	`feat/graph`	Parse every IAM policy from LocalStack → full `DiGraph` → `graph.graphml`

Next Ticket	Branch	First Commit
Scoring Engine	`feat/scoring`	Implement reachability, PageRank, hop-count → `risk_score` + `attack_path`
UI Polish / Eval / Report	later	Filters, export, evaluation notebook, final PDF, slides, demo script

Immediate actions for the team (today):

1. **Merge the seed PR** (`feat/terraform-seed` → `main`).
2. Create `feat/scanner` branch, start implementing real `boto3` enumeration in `scanner/main.py`.
3. Pair-program the scanner (2-person team) – target a working `make scan` by end of Week 3.
4. Keep the PR workflow: push → CI → review → squash-merge.

---

7. Tools & Environment Required

Tool	Version (tested)	Install Command
Docker Desktop / Docker Engine	24+	`brew install --cask docker` (macOS)
Docker Compose v2	bundled	included
Terraform	1.15+	`brew tap hashicorp/tap && brew install hashicorp/tap/terraform`
Python	3.11	`brew install python@3.11`
Python packages	`flake8`, `pytest`, `boto3`, `networkx`, `pandas`, `streamlit`, `streamlit-agraph`	`pip install -r scanner/requirements.txt` (similar for other services)
LocalStack	1.4.0 (Docker image)	`docker run -d --name localstack -p 4566:4566 -p 4571:4571 -e SERVICES=s3,iam,lambda,sts,ec2,kms,dynamodb -e START_WEB=0 localstack/localstack:1.4.0`
jq (for JSON inspection)	any	`brew install jq`
Git	any	`brew install git`

All commands are wrapped in the `Makefile`; running `make seed && make scan && make graph && make score && make ui` executes the full pipeline once the real implementations are in place.

---

8. How to Run the Dashboard (Live Performance)

```
# 1. Start LocalStack (once)
docker start localstack

# 2. Export env vars (or keep a .env file)
export ENDPOINT_URL=http://127.0.0.1:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1

# 3. Build & start the four micro-services
docker compose up -d

# 4. Run the full pipeline (once real scanner/graph/scorer exist)
make seed && make scan && make graph && make score

# 5. Open the Streamlit UI
make ui          # opens http://localhost:8501
```

The UI shows: * **Risk-Ranked Table** – sortable by `risk_score`. Click a row → highlights the exact attack path in the graph. * **Attack Graph** tab – interactive graph (`streamlit-agraph`) where nodes are IAM principals / resources and edges are *can-access* / *can-escalate* relations.

---

9. Summary for Teammates (Zero-Knowledge Friendly)

*We are building a tool that tells a cloud admin **which misconfigurations are actually dangerous** by drawing a map of who can do what in the cloud and finding the shortest path to admin privileges.*
The project is split into five micro-services (seed, scanner, graph, scorer, UI) that run in Docker.
All code lives in a single GitHub repo with CI that blocks bad code.
Right now the infrastructure seed works; the next two weeks we will write the real scanner, then the graph, then the scoring, then polish the UI and write the final report.

Prepared by the ISP team – ready for Friday evaluation.